개인정보보호암호기술

블록 암호 운영 모드

강원대학교 이세영

KNU 강원대학교 KANGWON NATIONAL UNIVERSITY

SDU 데이터보안활용 혁신융합대학 사업단

이번 시간에 다룰 내용

학습목표

01	패딩이 필요한 이유와 대표적인 패딩 방식을 설명할 수 있다
02	블록 암호를 사용하는 두 가지 방식을 구분할 수 있다
03	ECB · CBC · CFB · OFB · CTR 모드의 동작을 설명할 수 있다
04	다섯 가지 모드를 같은 기준으로 비교할 수 있다
05	(참고) 인증 암호(AEAD)의 개념과 CCM · GCM의 차이를 이해한다

Contents

01 **블록 암호로 긴 데이터 암호화하기**
블록 단위 처리, 블록 크기, 해결해야 할 두 가지

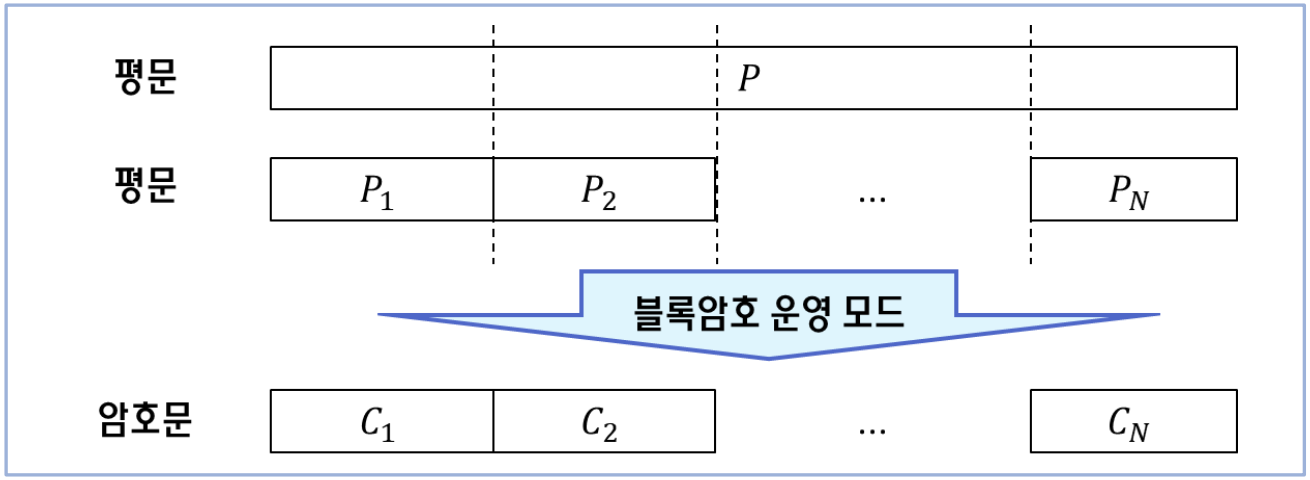
02 **패딩 (Padding)**
패딩이 필요한 이유, 제로 · 비트 · 바이트 패딩, PKCS#7

03 **운영 모드의 개요**
운영 모드란, 블록 암호를 쓰는 두 가지 방식, 초기 벡터

04 **다섯 가지 운영 모드**
ECB · CBC · CFB · OFB · CTR의 동작과 특성, 모드 비교

05 **Appendix : 인증 암호 (AEAD)**
암호화가 보장하지 않는 것, AEAD와 AAD, CCM과 GCM

긴 데이터는 블록으로 나누어 처리한다



블록 암호	블록 크기	예 : 1,000바이트를 나누면
DES	64비트 (8바이트)	8바이트 × 125개 (남는 바이트 없음)
AES	128비트 (16바이트)	16바이트 × 62개 + 8바이트가 남음 **(패딩 필요)**

블록 암호 = 블록 하나를 바꾸는 함수 / 운영 모드 = 여러 블록을 이어서 처리하는 방법

블록으로 나눌 때 정해야 할 두 가지

① 마지막 블록을 어떻게 채울 것인가

- **평문 길이가 블록 크기의 배수가 아니면 마지막 블록이 남음**

– 예 : AES로 1,000바이트 → 마지막에 8바이트만 남음

- **블록 암호는 정해진 크기의 블록만 처리 가능**

– → 남은 자리를 채우는 작업 : 패딩 (Padding)

② 블록들을 어떻게 이어서 처리할 것인가

- **각 블록을 따로 처리할 것인가**
- **앞 블록의 결과를 다음 블록에 섞을 것인가**
- **블록 암호로 만든 값을 평문과 결합할 것인가**

– → 이어 붙이는 방식의 규칙 : 운영 모드

방식에 따라 달라지는 것

- **처리 속도 (여러 블록을 동시에 처리할 수 있는가)**
- **전송 중 오류가 생겼을 때 영향을 받는 범위**
- **암호문이 가지는 성질 (같은 평문이 같은 암호문이 되는가)**

각 방식이 어떤 성질을 가지는지는 모드를 하나씩 살펴보며 확인

패딩이 필요한 이유

블록 크기 8바이트, 마지막 데이터가 5바이트인 경우



채우는 것으로 끝이 아니다

- 복호화 후 원래 평문 데이터를 인식해야 함

– 원본이 5바이트였다면 정확히 5바이트

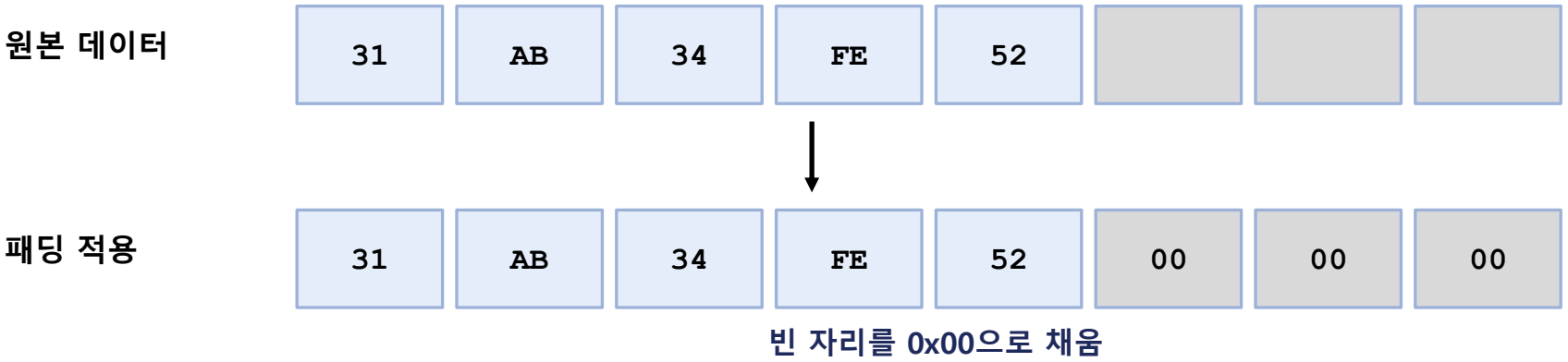
- 채운 부분을 그대로 돌려주면 데이터가 달라짐

좋은 패딩의 조건

- 채운 부분을 명확히 구분할 수 있어야 함
- 원래 데이터의 값과 무관하게 동작해야 함
- 송신자와 수신자가 같은 규칙을 사용해야 함

패딩 = 마지막 블록을 채우는 작업 + 복호화 후 정확히 걷어내는 규칙

패딩 방식 ① : 제로 패딩



문제 : 원래 데이터가 0x00으로 끝나면?

- **원본이 31 AB 34 00 인 경우**

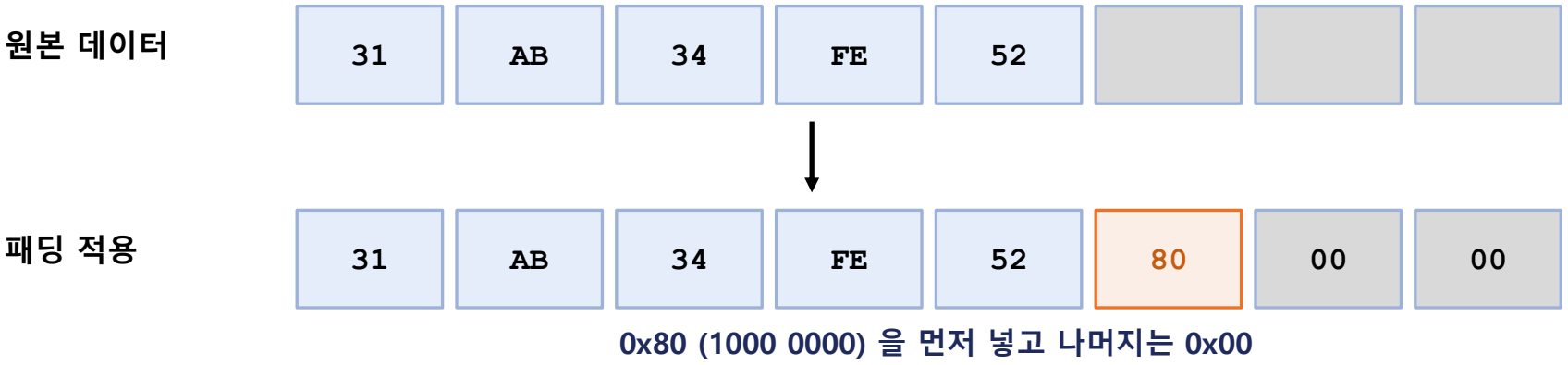
– 마지막 00이 데이터인지 채운 값인지 구분 불가

– 뒤쪽 0을 모두 지우면 원래 데이터까지 삭제됨

- **0x00으로 끝나지 않는 데이터에만 제한적으로 사용 가능**

구현은 가장 쉽지만, "데이터 값과 무관하게 동작해야 한다"는 조건을 만족하지 못함

패딩 방식 ② : 비트 패딩



0x80 (1000 0000) 을 먼저 넣고 나머지는 0x00

패딩 제거 방법

- 뒤에서부터 0x00을 지나
- 처음 만나는 0x80까지 제거

– 원래 데이터가 0x00으로 끝나도 경계가 분명

주의할 점

- 원래 데이터가 0x80으로 끝나도 문제없음

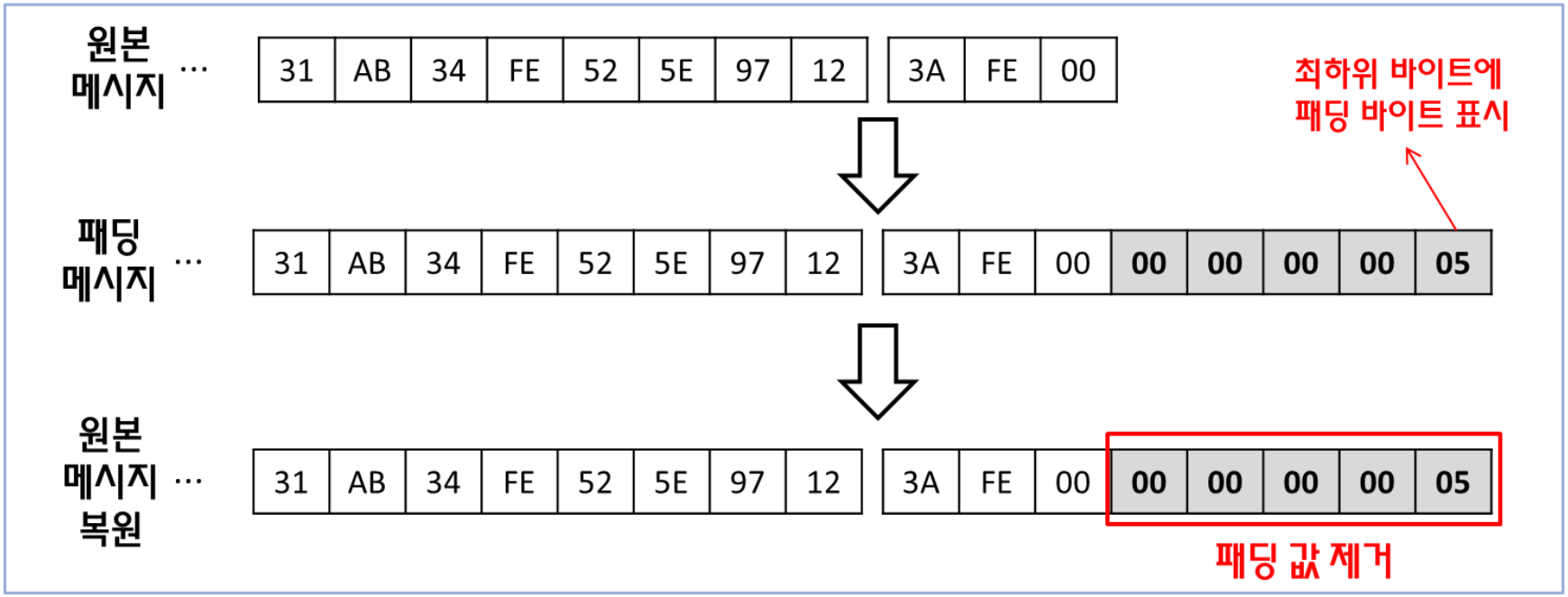
– 뒤에서 처음 만나는 0x80 하나만 제거

- 평문이 이미 블록 크기의 배수이면

– 블록 하나를 더 붙여 패딩을 넣음

비트 패딩 = "1을 하나 넣고 나머지는 0" → 데이터 값과 무관하게 경계를 구분할 수 있음

패딩 방식 ③ : 바이트 패딩

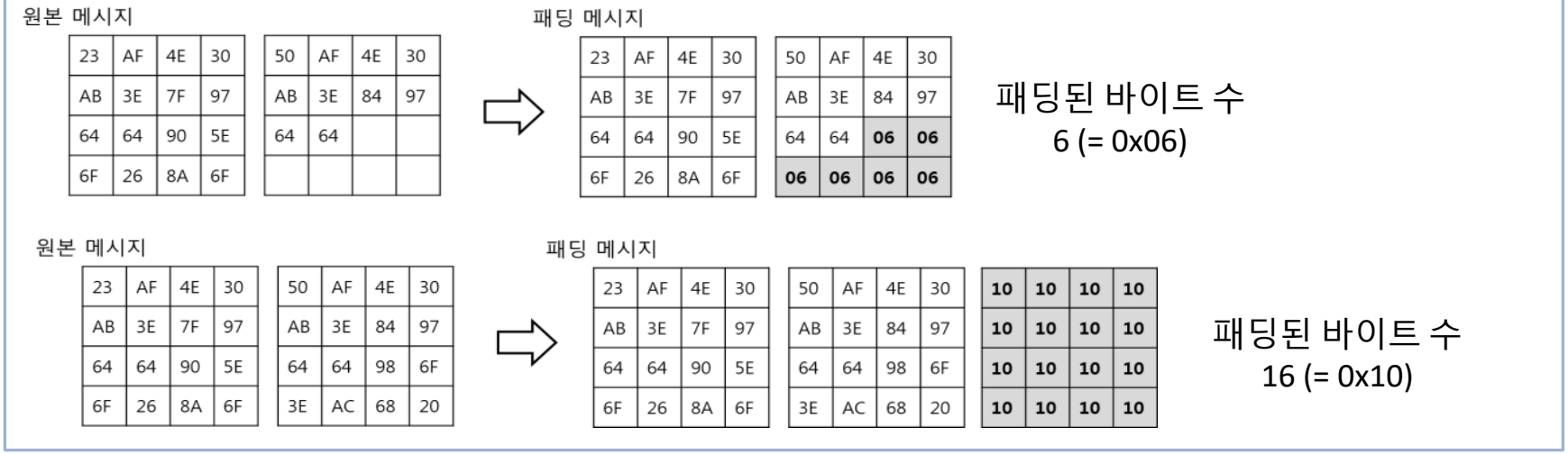


규칙과 제거 방법

- **빈 자리를 채우고, 마지막 바이트에 "채운 바이트 수"를 기록**
- **복호화 후 마지막 바이트 값만큼 뒤에서 제거**

– 그림의 예 : 5바이트를 채웠으므로 마지막 바이트는 05

패딩 방식 ④ : PKCS#7 패딩



규칙

- **채울 바이트 수가 n이면 값 n을 n개 채움**

– 6바이트 부족 → 06 06 06 06 06 06

- **복호화 : 마지막 바이트 값만큼 뒤에서 제거**

– 지우는 바이트가 모두 같은 값인지 확인 가능

평문이 이미 블록 크기의 배수이면

- **패딩을 생략하지 않고 블록 하나를 추가**

– 16바이트 블록 → 0x10을 16개 채운 블록 추가

- **생략하면 수신자가 데이터를 잘못 잘라낼 수 있음**

PKCS#7 = "채울 바이트 수를 그 값으로 채운다"

패딩 방식의 비교

방식	채우는 값	제거 방법	특징
제로 패딩	0x00	뒤쪽 0x00 제거	가장 단순하나 데이터가 00으로 끝나면 구분 불가
비트 패딩	0x80 + 0x00	뒤에서 첫 0x80까지 제거	데이터 값과 무관하게 동작
바이트 패딩	0 + 마지막에 개수	마지막 바이트 값만큼 제거	제거가 빠름
PKCS#7	개수 n을 n개	마지막 바이트 값만큼 제거	가장 널리 사용, 패딩 형태 검사도 쉬움

공통 규칙

- **평문이 이미 블록 크기의 배수여도 블록 하나를 더 붙임**
- **송신자와 수신자가 같은 규칙을 사용해야 함**

패딩은 평문을 블록 단위로 맞추기 위한 작업 → 평문을 블록 암호에 직접 넣는 방식에서만 필요

운영 모드 (Mode of Operation) 란

ECB	CBC	CFB	OFB	CTR
Electronic Codebook	Cipher Block Chaining	Cipher Feedback	Output Feedback	Counter
전자 코드북	암호 블록 연쇄	암호 피드백	출력 피드백	카운터

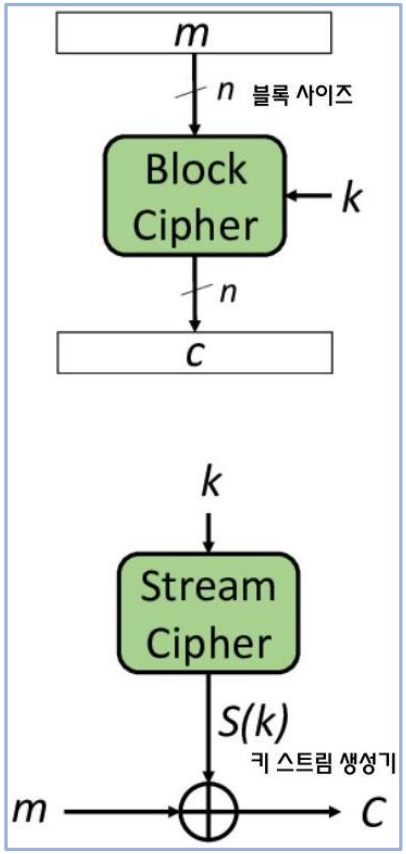
NIST SP 800-38A 가 제시하는 다섯 가지 기본 운영 모드

각 모드를 볼 때 확인할 네 가지

- ① 블록을 어떻게 이어 붙이는가 (구조)
- ② 패딩이 필요한가
- ③ 여러 블록을 동시에 처리할 수 있는가 (병렬 처리)
- ④ 전송 중 오류가 생기면 어디까지 영향을 받는가 (오류 전파)

같은 블록 암호를 쓰더라도 운영 모드에 따라 암호문의 성질과 처리 방식이 달라진다

블록 암호를 사용하는 두 가지 방식



방식 ① 평문 블록을 블록 암호에 직접 넣는다

- **평문이 블록 암호를 통과 → 블록 크기에 맞춰야 함**

– → 패딩 필요

- **복호화에는 블록 암호의 복호화 함수가 필요**

– 해당 모드 : ECB, CBC

방식 ② 블록 암호로 만든 값을 평문과 XOR한다

- **블록 암호는 키 스트림을 만드는 데 사용**
- **평문은 블록 암호를 통과하지 않고 XOR만 됨**

– → 패딩 불필요 (암호문 길이 = 평문 길이)

– → 복호화에도 암호화 함수만 사용

– 해당 모드 : CFB, OFB, CTR

두 방식 모두 블록 암호를 사용하지만, 평문이 블록 암호를 "통과하는가"가 다르다

초기 벡터 (IV) 의 역할

초기 벡터가 없다면

- **블록 암호는 같은 입력 → 항상 같은 출력**
- **같은 데이터를 두 번 암호화하면 암호문이 동일**

– 두 메시지가 같은지 공격자가 알 수 있음
– 같은 값을 가진 데이터가 그대로 드러남

초기 벡터의 역할

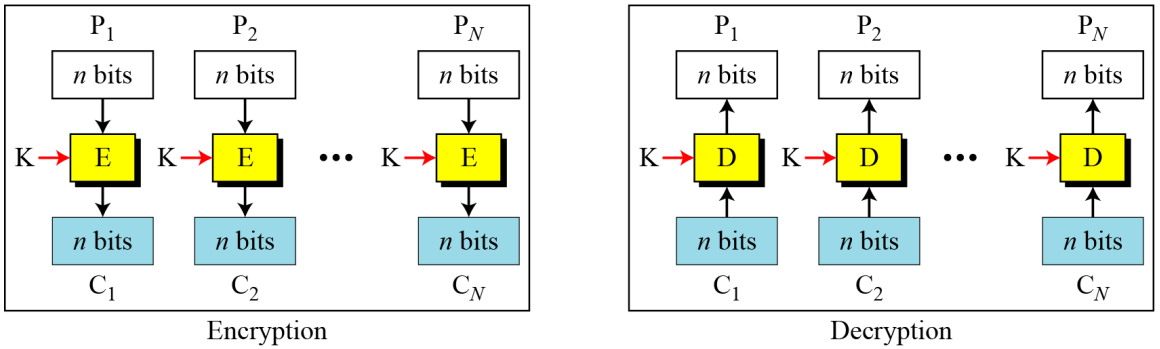
- **암호화할 때마다 달라지는 값을 하나 더 넣음**
- **같은 평문을 암호화해도 매번 다른 암호문**

– → 암호문만 보고는 같은 데이터인지 알 수 없음

성질	내용
비밀 값이 아님	복호화 시 필요하므로 암호문과 함께 전달 · 저장 (앞에 붙이는 것이 일반적)
매번 달라야 함	같은 키로 같은 IV를 다시 사용하면 안 됨
예측 불가 조건	모드에 따라 다음 IV를 예측할 수 없어야 함
만드는 방법	암호용 안전한 난수 발생기로 생성

IV = 같은 평문을 매번 다른 암호문으로 만들기 위한 값 (ECB를 제외한 모든 모드에서 사용)

ECB (Electronic Codebook) 모드



왼쪽 : 암호화 / 오른쪽 : 복호화 — 블록마다 독립적으로 처리

동작

- $C_i = E_K(P_i)$
- $P_i = D_K(C_i)$
- 블록 사이에 연결 없음

특성

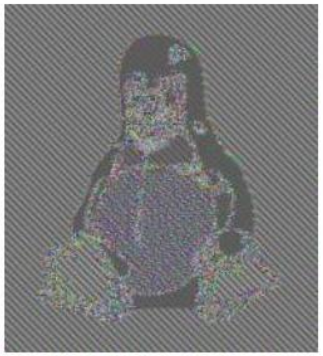
- 패딩 필요 / IV 사용 안 함
- 암호화 · 복호화 모두 병렬 처리 가능
- 원하는 블록만 복호화 가능
- 오류는 해당 블록에만 영향

같은 평문 블록 → 항상 같은 암호문 블록 : 이 성질이 어떤 결과를 낳는지 확인해 보자

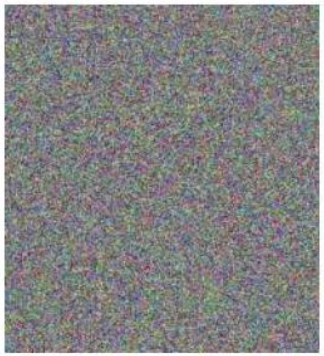
ECB의 한계 ① : 데이터의 패턴이 남는다



원본



ECB 모드로 암호화



다른 모드로 암호화

왜 패턴이 남는가

- 같은 색이 이어지는 영역
= 같은 평문 블록
- ECB : 같은 평문 블록
→ 같은 암호문 블록

– 색만 바뀌고 모양은 그대로

개인정보 처리에서의 의미

- **DB 컬럼을 ECB로 암호화하면 같은 값 → 같은 암호문**

– 누구와 누가 같은 값인지 드러남

– 값의 종류가 적으면(성별 · 등급 등) 개수를 세어 값 추측 가능

암호화를 했는데도 데이터의 패턴이 그대로 남는다

ECB의 한계 ② : 블록을 복사해 붙일 수 있다

상황

- **이름 · 점수 표에서 점수 컬럼만 ECB로 암호화**
- **공격자 이브는 키를 모름**

– 다만 앨리스의 점수가 더 높다는 사실은 알고 있음

공격

- **앨리스 행의 암호문을 복사해 자신의 행에 덮어쓰기**
- **복호화 결과 : 이브의 점수가 90점이 됨**

– 키를 몰라도 데이터를 바꿀 수 있음

- **블록 잘라내기 · 순서 바꾸기도 가능**

이름	암호화된 점수 (원본 점수)
Alice	0F14D3F2 (90)
Bob	3DE9001F (80)
Eve	549F2D4F (50)

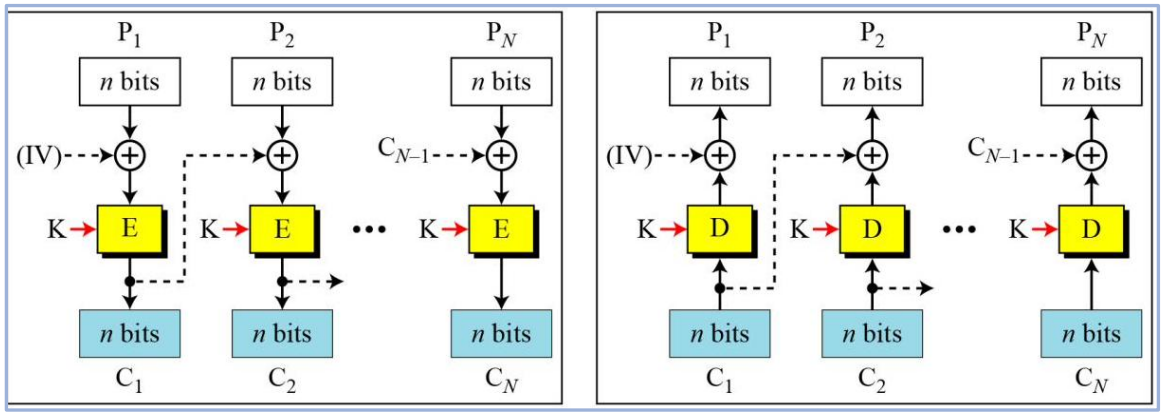


이름	암호화된 점수 (원본 점수)
Alice	0F14D3F2 (90)
Bob	3DE9001F (80)
Eve	**0F14D3F2 (90)**

덮어쓰기

ECB는 패턴이 남고 블록 단위 조작이 쉬움 → 여러 표준과 지침에서 단독 사용을 권하지 않음

CBC (Cipher Block Chaining) 모드



왼쪽 : 암호화 (앞 암호문과 XOR 후 암호화) / 오른쪽 : 복호화

동작

- **암호화 : $C_i = E_K(P_i \oplus C_{i-1})$**

– C_0 자리에 초기 벡터(IV) 사용

- **복호화 : $P_i = D_K(C_i) \oplus C_{i-1}$**

기본 성질

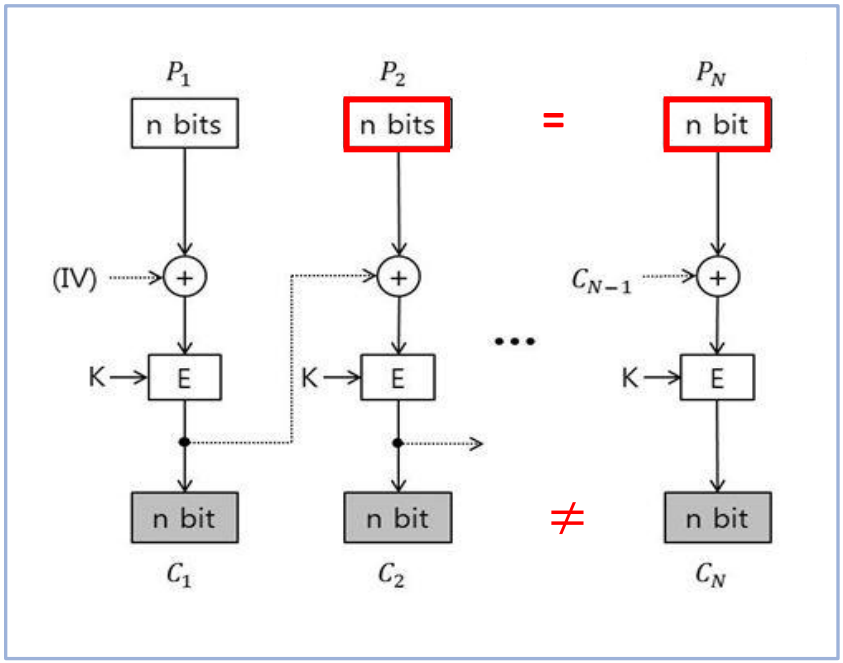
- **평문 블록을 블록 암호에 직접 입력**

– → 패딩 필요

- **복호화에 D 함수 필요**
- **초기 벡터 사용**

CBC = 앞 블록의 암호문을 다음 블록에 섞어 블록들을 사슬처럼 연결

CBC 모드의 특성 ① : 연결성과 병렬 처리



같은 평문 블록($P_2 = P_n$)이라도 암호문 블록은 서로 다름

연결성 덕분에 같은 평문 블록도 다른 암호문이 됨 (ECB의 패턴 문제 해결)

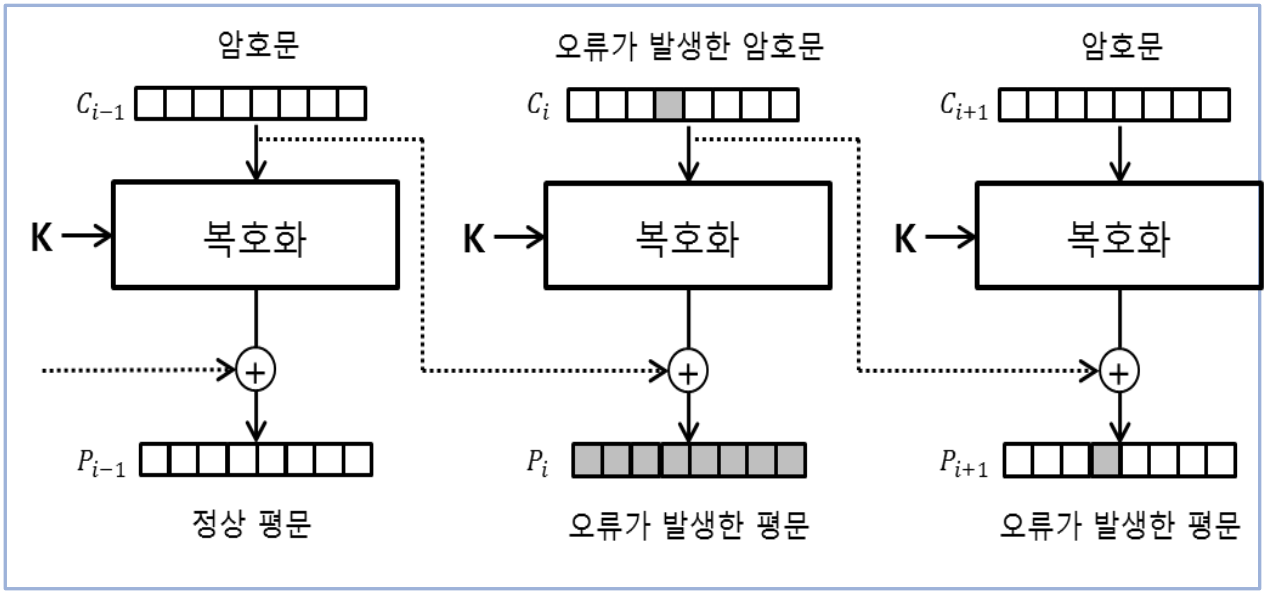
병렬 처리

- **암호화 : 앞 블록의 암호문이 나와야 다음 블록 처리 가능**
  - → 순서대로만 처리 (병렬 불가)
- **복호화 : 필요한 앞 암호문을 이미 모두 가지고 있음**
  - → 여러 블록 동시 처리 가능 (병렬 가능)

초기 벡터 조건

- **IV가 매번 같으면 첫 블록의 패턴이 드러남**
- **공격자가 다음 IV를 예측할 수 있으면**
  - 자신이 고른 평문을 넣어 특정 값을 확인할 수 있음
  - → 매번 다르고 예측 불가능한 난수 IV 필요

CBC 모드의 특성 ② : 오류가 번지는 범위

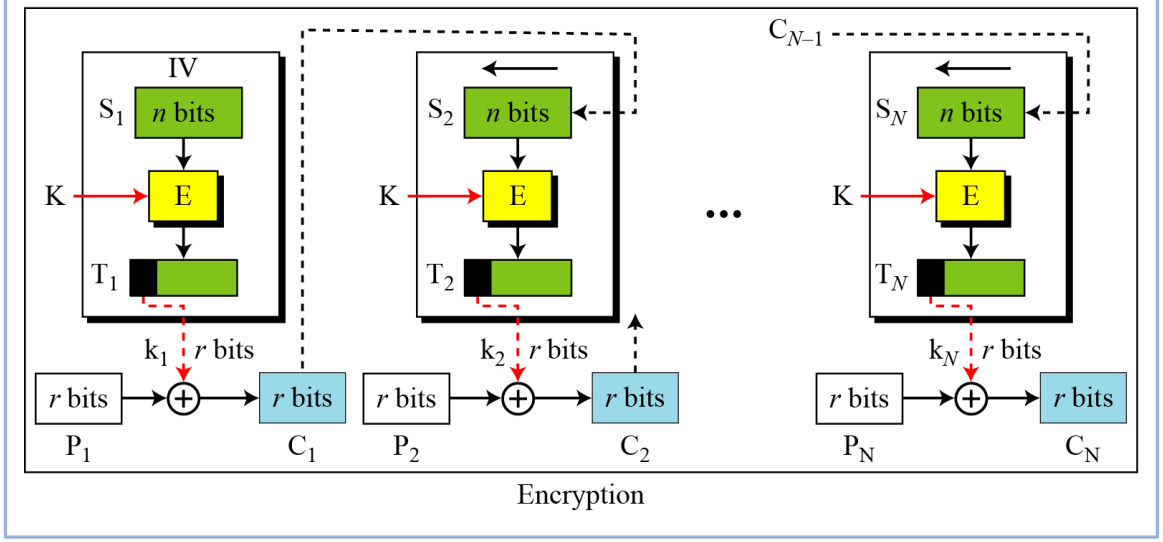


오류가 번지는 범위

- **P_i : 블록 전체 손상**
  - 블록 암호는 입력 1비트만 달라져도 출력이 완전히 달라짐
- **P_{i+1} : 같은 위치 1비트만 반전**
- **P_{i+2} 이후 : 정상 복호화**
  - → 오류가 두 블록에서 멈춤 (자기 복구; Self-recovering)

C_i의 1비트 오류 → P_i 전체 손상 + P_{i+1}의 같은 위치 1비트 반전

CFB (Cipher Feedback) 모드



앞 블록의 암호문을 블록 암호의 입력으로 되먹임

기본 성질

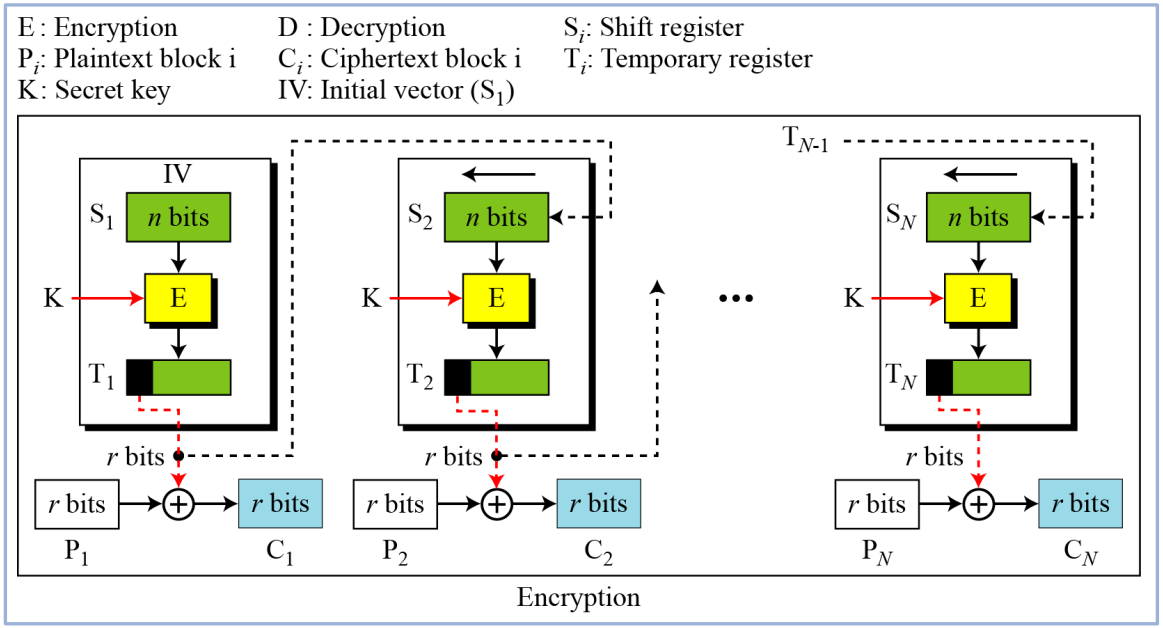
- 블록 암호 출력과 평문을 XOR
  - 패딩 불필요, 복호화에도 E 함수만 사용
- 블록보다 작은 단위(바이트 등)로도 처리 가능

병렬 처리와 오류

- 암호화 : 순서대로만 가능 (병렬불가)
- 복호화 : 병렬 가능
- 오류 : 해당 블록 1비트 + 쉬프트 레지스터에서 오류가 사라질 때 까지 다음 블록 전체

CFB = 암호문을 되먹여 다음 블록의 입력으로 사용 (블록 암호를 스트림 암호처럼 사용)

OFB (Output Feedback) 모드



블록 암호의 "출력"을 다음 입력으로 되먹임 (평문 · 암호문과 무관)

기본 성질

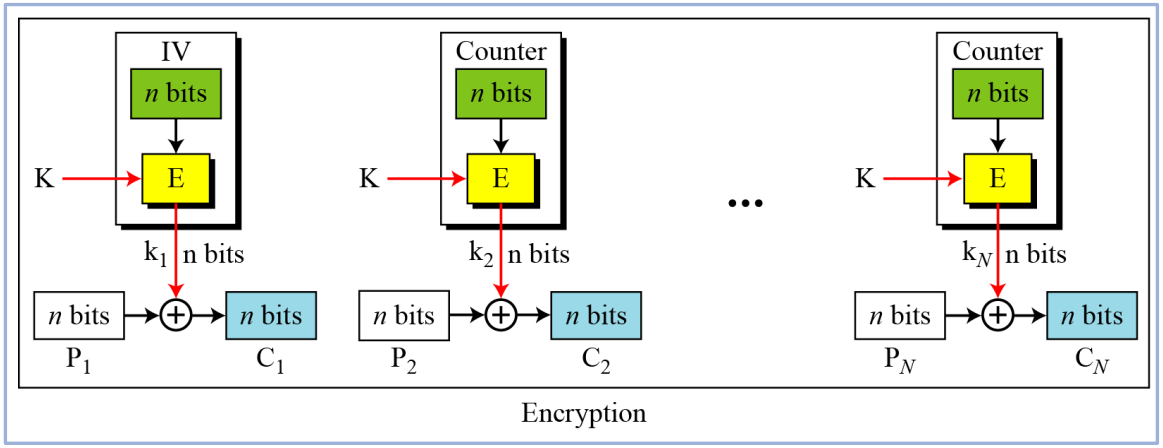
- 키 스트림이 평문과 무관 → 미리 계산 가능(전처리)
- 오류는 해당 비트에만 영향 (번지지 않음)
- 패딩 불필요, 복호화에도 E 함수만 사용
- 병렬 처리 : 암호화 · 복호화 모두 불가

주의

- 같은 키 + 같은 IV → 같은 키 스트림
  - 두 암호문을 XOR하면 두 평문의 XOR이 드러남

OFB = 블록 암호의 출력만 이어 붙여 키 스트림을 만듦 → 오류가 번지지 않음

CTR (Counter) 모드



되먹임 없음 : 블록마다 카운터 값을 암호화해 평문과 XOR

기본 성질

- 키 스트림이 카운터 값만으로 결정
  - 암호화 · 복호화 모두 병렬 처리 가능
- 원하는 위치의 블록만 골라 처리 가능
- 패딩 불필요, 복호화에도 E 함수만 사용
- 오류는 해당 비트에만 영향

주의

- 같은 키 + 같은 카운터 값 재사용 금지
  - 키 스트림이 그대로 재사용됨

CTR = 카운터 값을 암호화해 키 스트림을 만듦 → 병렬 처리와 임의 위치 접근이 쉬움

다섯 가지 모드의 비교 ① : 구조와 패딩

구분	ECB	CBC	CFB	OFB	CTR
블록 암호 사용 방식	평문을 직접 암호화	평문을 직접 암호화	만든 값과 XOR	만든 값과 XOR	만든 값과 XOR
블록을 잇는 방법	연결 없음	앞 암호문을 평문에 XOR	앞 암호문을 입력으로	출력을 다시 입력으로	카운터 값 사용
패딩	**필요**	**필요**	불필요	불필요	불필요
초기 벡터	**사용 안 함**	사용	사용	사용	카운터 시작값
복호화에 D 함수	필요	필요	불필요	불필요	불필요

패딩 · 복호화 함수의 필요 여부는 모두 "평문이 블록 암호를 통과하는가"에서 나온다

다섯 가지 모드의 비교 ② : 병렬 처리와 오류

구분	ECB	CBC	CFB	OFB	CTR
암호화 병렬 처리	○	×	×	×	○
복호화 병렬 처리	○	○	○	×	○
원하는 블록만 처리	○	×	×	×	○
전송 오류의 영향	P_i 전체	P_i 전체 + P_{i+1} 1비트	P_i 1비트 + P_{i+1} 이후 쉬프트 레지스터 나갈 때까지 전체	해당 비트	해당 비트

선택할 때 보는 점

- 큰 파일의 임의 위치 접근 · 빠른 처리 → 카운터 방식
- 전송 오류가 잦은 환경 → 오류가 번지지 않는 방식(OFB · CTR)
- 블록 단위 패턴이 그대로 남는 ECB는 제외

같은 블록 암호라도 운영 모드에 따라 처리 방식과 오류 특성이 달라진다

[Appendix] 암호화가 보장하는 것과 보장하지 않는 것

암호화가 보장하는 것

- **키가 없으면 내용을 읽을 수 없음**

– = 기밀성 (Confidentiality)

암호화가 보장하지 않는 것

- **키가 없어도 암호문을 바꿀 수 있음**

– 바뀐 사실을 수신자가 모를 수 있음

- **순서 바꾸기 · 잘라내기도 막지 못함**

공격자 : 내용을 읽으려 함 → **암호화가 막아 준다**

공격자 : 내용을 바꾸려 함 → **암호화는 막아 주지 못한다**

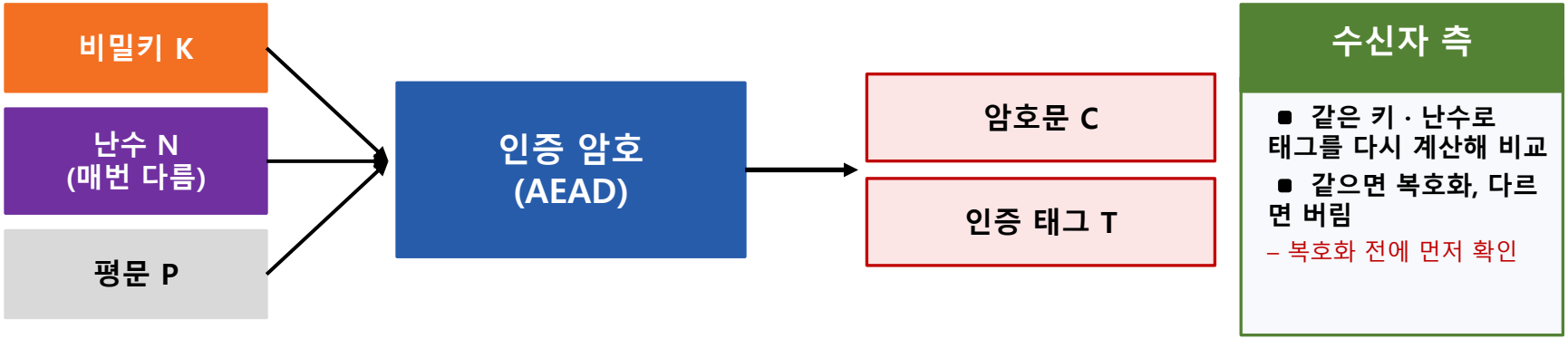
앞에서 본 예

- **ECB 모드에서 점수 블록을 복사해 덮어쓴 경우**

– 키를 모르는 공격자가 데이터를 바꾸었지만 암호는 깨지지 않음

- **다른 모드에서도 형태만 다를 뿐 비슷한 조작이 가능**

[Appendix] 인증 암호 (AEAD) 의 개념



인증 암호를 쓰는 이유

- 기밀성(내용 숨기기)과 무결성(바뀌었는지 확인)을 한 번에 제공
- 두 기능을 따로 구현하면서 생기는 실수를 줄일 수 있음
- 통신 프로토콜과 저장 암호화에서 널리 사용

[Appendix] 추가 인증 데이터 (AAD)

헤더 (수신자 주소, 메시지 번호, 버전)	본문 (보호할 내용)
숨길 수 없음 (중간 장비가 읽어야 함) 그러나 바뀌면 안 됨	숨겨야 하고, 바뀌면 안 됨

입력	암호화	인증(태그에 반영)	예
추가 인증 데이터 (AAD)	**하지 않음**	**함**	헤더, 수신자 식별자, 버전
평문	**함**	**함**	메시지 본문, 저장할 데이터

결과

- **인증 태그는 암호문과 추가 인증 데이터 양쪽을 모두 반영**
- **둘 중 어느 하나라도 1비트가 바뀌면 검증 실패**

– → 숨길 필요는 없지만 바뀌면 안 되는 정보를 함께 보호

[Appendix] 대표적인 인증 암호 : CCM과 GCM

CCM (Counter with CBC-MAC)

- **암호화 : 카운터 모드**
- **인증 : 블록을 사슬로 이어 가며 태그 생성**

– 태그 계산은 순서대로만 가능

- **블록당 블록 암호 연산 2회 (암호화 + 인증)**
- **무선랜 보안 규격 등에서 사용**

GCM (Galois/Counter Mode)

- **암호화 : 카운터 모드**
- **인증 : 유한체 곱셈으로 태그 생성**

– 여러 블록을 동시에 계산 가능 → 빠름

- **하드웨어 가속이 잘 적용됨**
- **웹 통신 등 고속 환경에서 널리 사용**

구분	CCM	GCM
암호화 방식	카운터 모드	카운터 모드
태그 생성 방식	블록 사슬 방식 (순차)	유한체 곱셈 (병렬 가능)
속도	상대적으로 느림	빠름
추가 인증 데이터	지원	지원
난수 재사용	**금지**	**금지**

두 방식 모두 암호화는 카운터 모드 — 차이는 인증 태그를 만드는 방법과 속도

핵심 정리 (1) : 패딩과 운영 모드

패딩	마지막 블록을 채우고, 복호화 후 정확히 제거할 수 있어야 함 PKCS#7 : 채울 바이트 수 n을 값 n으로 n개 (배수여도 한 블록 추가)
두 가지 사용 방식	평문을 블록 암호에 직접 입력 (ECB · CBC) → 패딩 필요, D 함수 필요 만든 값과 평문을 XOR (CFB · OFB · CTR) → 패딩 불필요, E 함수만 사용
모드별 요점	ECB : 연결 없음 / CBC : 앞 암호문을 평문에 XOR CFB : 앞 암호문을 입력으로 / OFB : 출력을 입력으로 / CTR : 카운터 값 사용
초기 벡터(IV)	같은 평문을 매번 다른 암호문으로 만들기 위한 값, 비밀은 아님 매번 달라야 함 (CBC는 예측 불가), 같은 키 + 같은 IV → 같은 키 스트림

핵심 정리 (2) : 모드 비교

병렬 처리	ECB · CTR : 암호화 · 복호화 모두 가능, 원하는 블록만 처리 가능 CBC · CFB : 암호화 불가, 복호화는 가능 / OFB : 둘 다 불가
전송 오류의 영향	ECB : 해당 블록 / CBC : 해당 블록 + 다음 블록 1비트 CFB : 해당 블록 1비트 + 쉬프트 레지스터에서 나갈 때 까지 블록 / OFB · CTR : 해당 비트
Appendix : 인증 암호	AEAD : 암호화 + 인증 태그를 한 번에 (AAD는 암호화하지 않고 인증만) CCM · GCM : 암호화는 카운터 모드, 태그 생성 방식이 다름 / 난수 재사용 금지

Thank You

E-mail: seyoung@kangwon.ac.kr